

ExpenseEase (Reimbursement Portal)

Project Documentation

Overview

ExpenseEase is a full-stack internal reimbursement management system where:

- **EMPLOYEE** can submit reimbursement claims and track their status.
- **MANAGER** can review claims assigned to them (approve/reject with comments).
- **ADMIN** manages users/roles and can oversee claims (including acting as fallback reviewer).

The system is implemented as:

- **Frontend**: Multi-page static UI (HTML/CSS/JS).
- **Backend**: REST API (Java 17, Spring Boot) with layered architecture (controller → service → repository).
- **Database**: PostgreSQL via Spring Data JPA / Hibernate.

Technology Used

Backend

- **Java**: 17
- **Spring Boot**: 4.0.6
- **Spring Modules**
 - Spring Web MVC (spring-boot-starter-webmvc)
 - Spring Data JPA (spring-boot-starter-data-jpa)
 - Spring Security (spring-boot-starter-security)
 - Jakarta Validation (spring-boot-starter-validation)
- **ORM**: Hibernate (via JPA)
- **Build**: Maven
- **DB**: PostgreSQL
- **Quality**: JaCoCo configured in Maven (jacoco-maven-plugin)

Frontend

- **HTML / CSS / JavaScript** (Multi-Page App)

Features and Functionalities

Authentication

- **Login**: POST /auth/login
 - Backend validates credentials using PasswordEncoder.matches() and returns identity payload (userId , name , email , role).
 - Frontend stores this in localStorage as userSession .
 - Subsequent API calls include the X-USER-ID header automatically .

User & Role Management (Admin)

- **Create user:** POST /users
- **List users:** GET /users
- **Filter users by role:** GET /users/by-role?roleParam=ADMIN|MANAGER|EMPLOYEE
- **Assign manager:** PUT /users/{userId}/assign-manager?managerId={managerId}
- **Delete user:** DELETE /users/{userId}

Rules enforced in service layer (UserService):

- Only **ADMIN** can create, list, filter, assign manager, delete.
- Company email validation is enforced in the backend using EmailValidator + app.allowed-email-domain.
- Passwords are stored hashed (BCrypt).

Claim Management

- **Submit claim:** POST /claims (EMPLOYEE only, must be own employeeId)
- **Get claim by id:** GET /claims/{claimId} (scoped by role/ownership/reviewer assignment)
- **Employee claim list:** GET /claims/user-claims
- **Employee paginated claims:**
GET /claims/my/paginated?page=&size=&sort=&claimStatus=...
- **Reviewer queue:** GET /claims/reviewer
- **Reviewer queue (paginated):** GET /claims/reviewer/paginated?...
- **Approve claim:** PUT /claims/{claimId}/approve?comments=... (MANAGER/ADMIN)
- **Reject claim:** PUT /claims/{claimId}/reject?comments=... (MANAGER/ADMIN)
- **Admin list all claims:** GET /claims/all
- **Admin list all claims (paginated):** GET /claims/all/paginated?...
- **Admin delete claim:** DELETE /claims/{claimId} (ADMIN only; only if status is SUBMITTED)

Claim Lifecycle

Status (ClaimStatus):

- SUBMITTED
- APPROVED
- REJECTED

Core rules enforced in service layer (ClaimService):

- Only **EMPLOYEE** can submit claims, and only for their own employeeId .
- Claim reviewer assignment:
 - If an employee has a managerId → reviewer is that manager.
 - Else → reviewer is “any admin” (fallback).
- Only **MANAGERS** assigned as reviewerId can approve/reject that claim.
- **ADMIN** can approve/reject any claim (and will overwrite reviewerId to themselves when they do).
- Claim amount validation:
 - Must be positive and finite
 - Must be $\leq$ app.max-claim-amount (default 50000)
- Claim date:
 - Defaults to today if not provided
 - Cannot be in the future

API response shape + error handling

- **Standard response format:**
 - success (boolean)
 - message (string)
 - data (payload)
- **Global exception mapping:** GlobalExceptionHandler converts common errors to consistent HTTP responses:
 - 400 (Bad request / validation)
 - 401 (Unauthorized)
 - 403 (Forbidden)
 - 404 (Not found)
 - 409 (Conflict / constraint violations)
 - 500 (Fallback)

DataBase Schemas

users table

Logical columns (from entity/User.java):

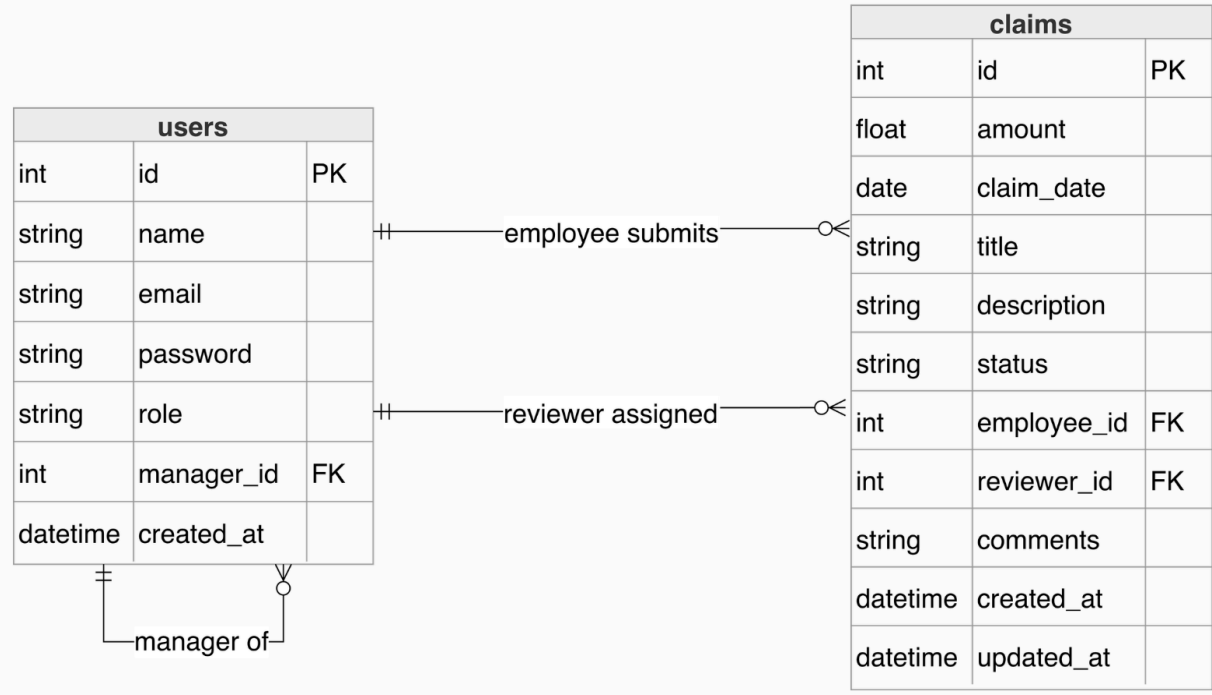
- id (PK, auto-increment)
- name
- email (unique at business level, enforced in service, not explicitly shown as DB constraint in entity)
- password (bcrypt hash)
- role (ADMIN|MANAGER|EMPLOYEE)
- manager_id (nullable, references another users.id conceptually)
- created_at

claims table

Logical columns (from entity/Claim.java):

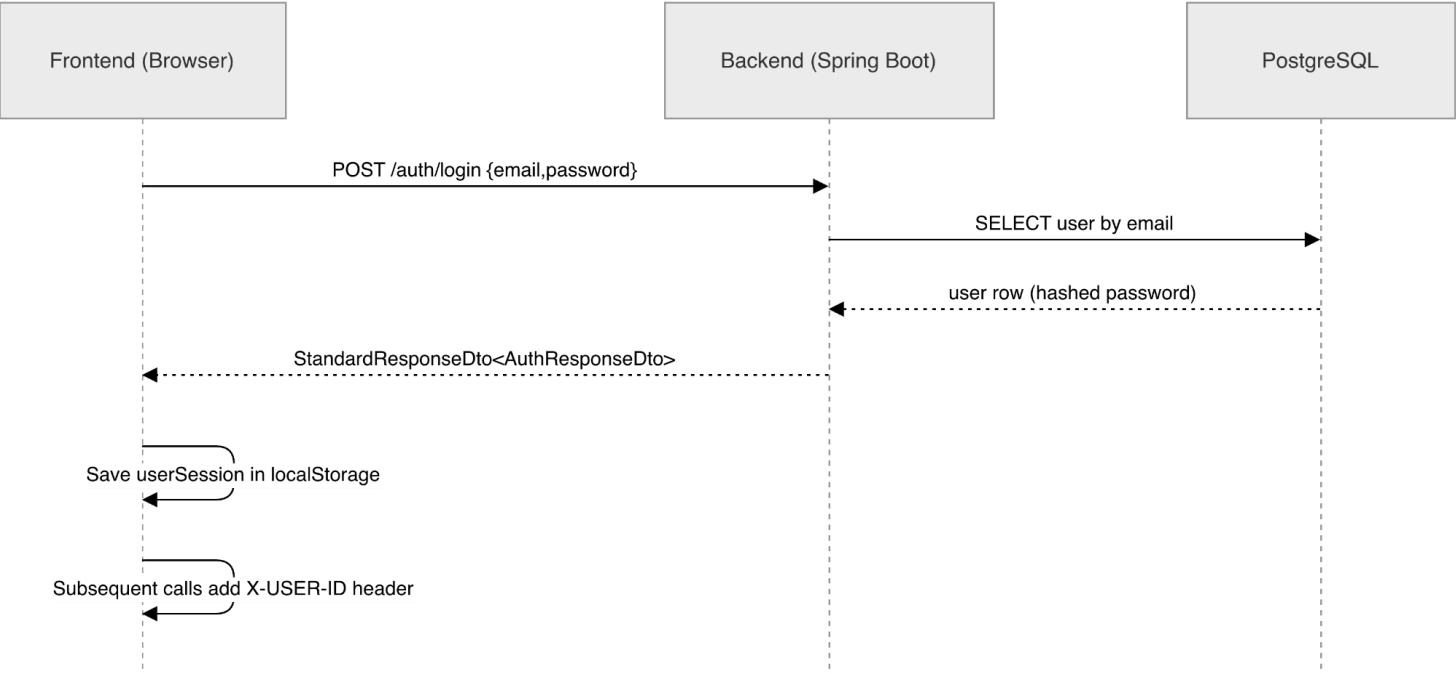
- id (PK, auto-increment)
- amount
- date
- title
- description
- status (SUBMITTED|APPROVED|REJECTED)
- employee_id (references users.id conceptually)
- reviewer_id (references users.id conceptually)
- comments (nullable)
- created_at
- updated_at (nullable)

ER Diagrams

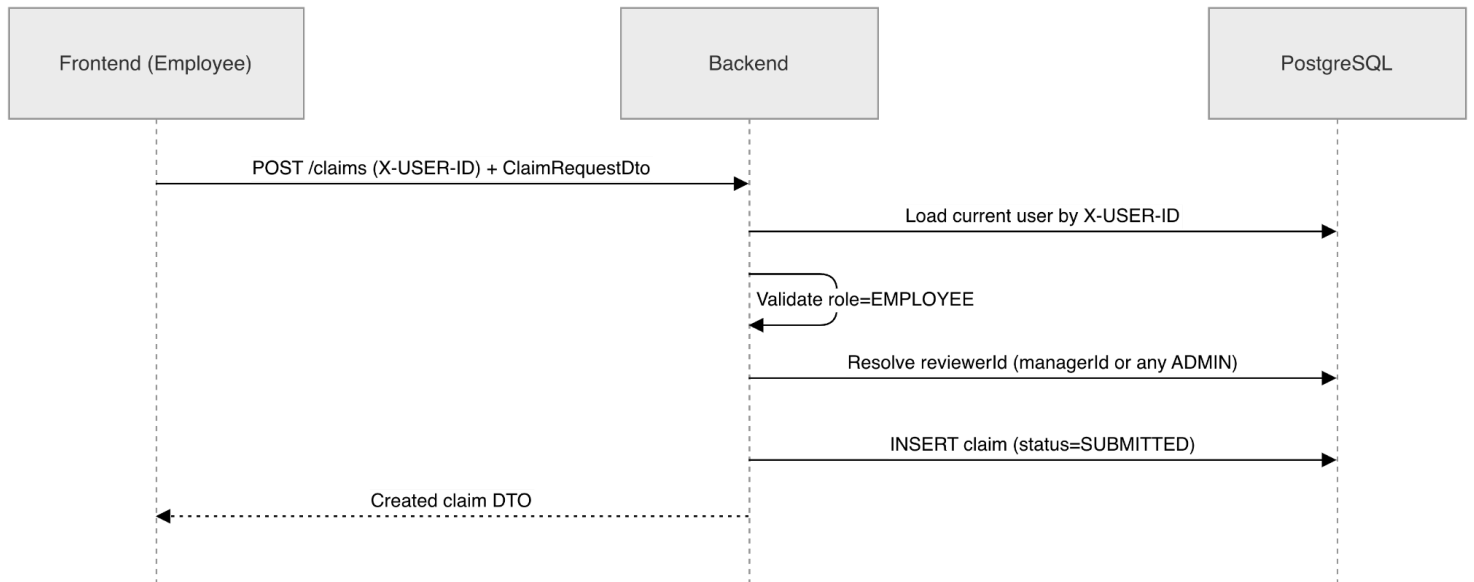


Workflows

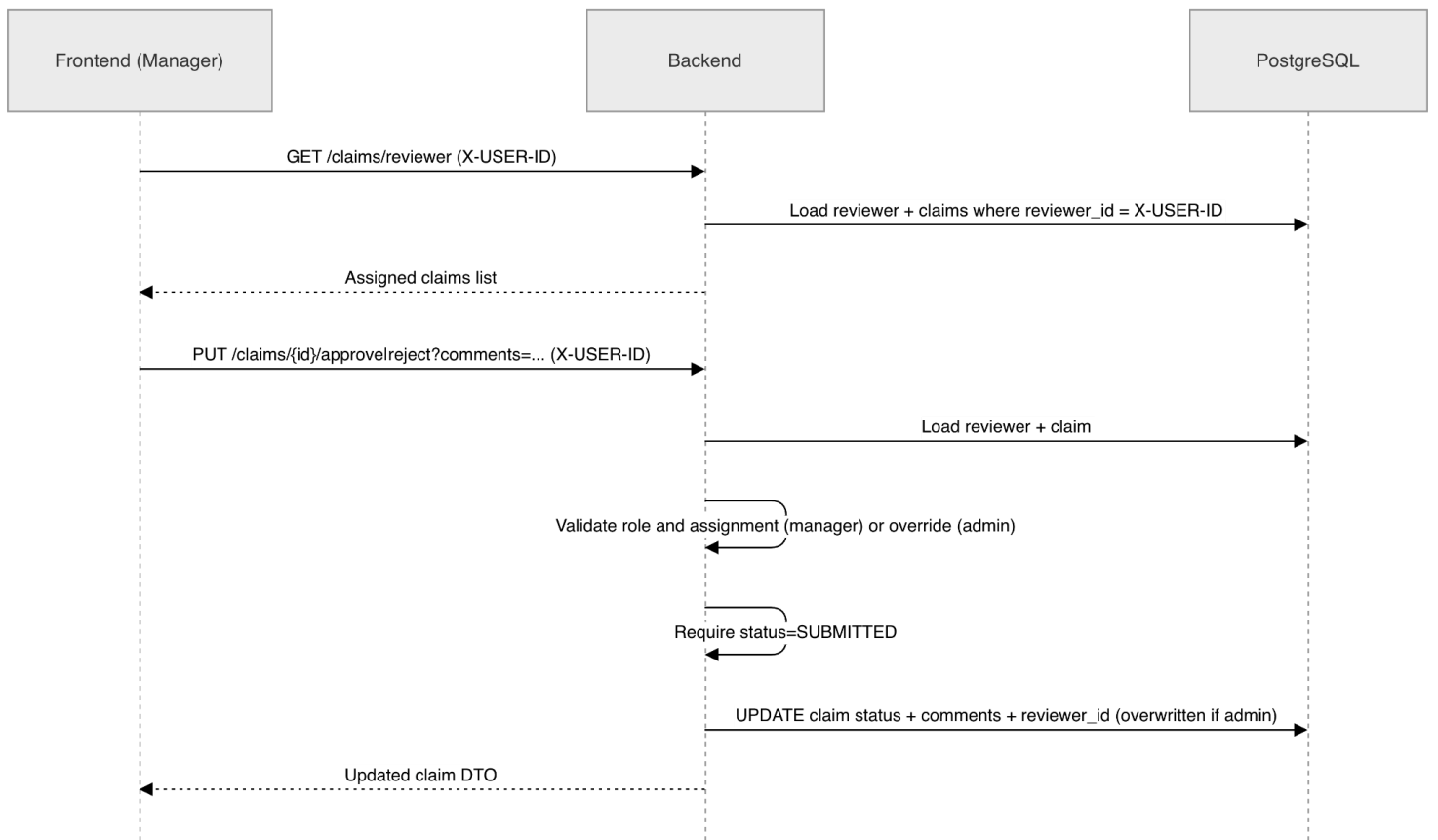
1) Login workflow



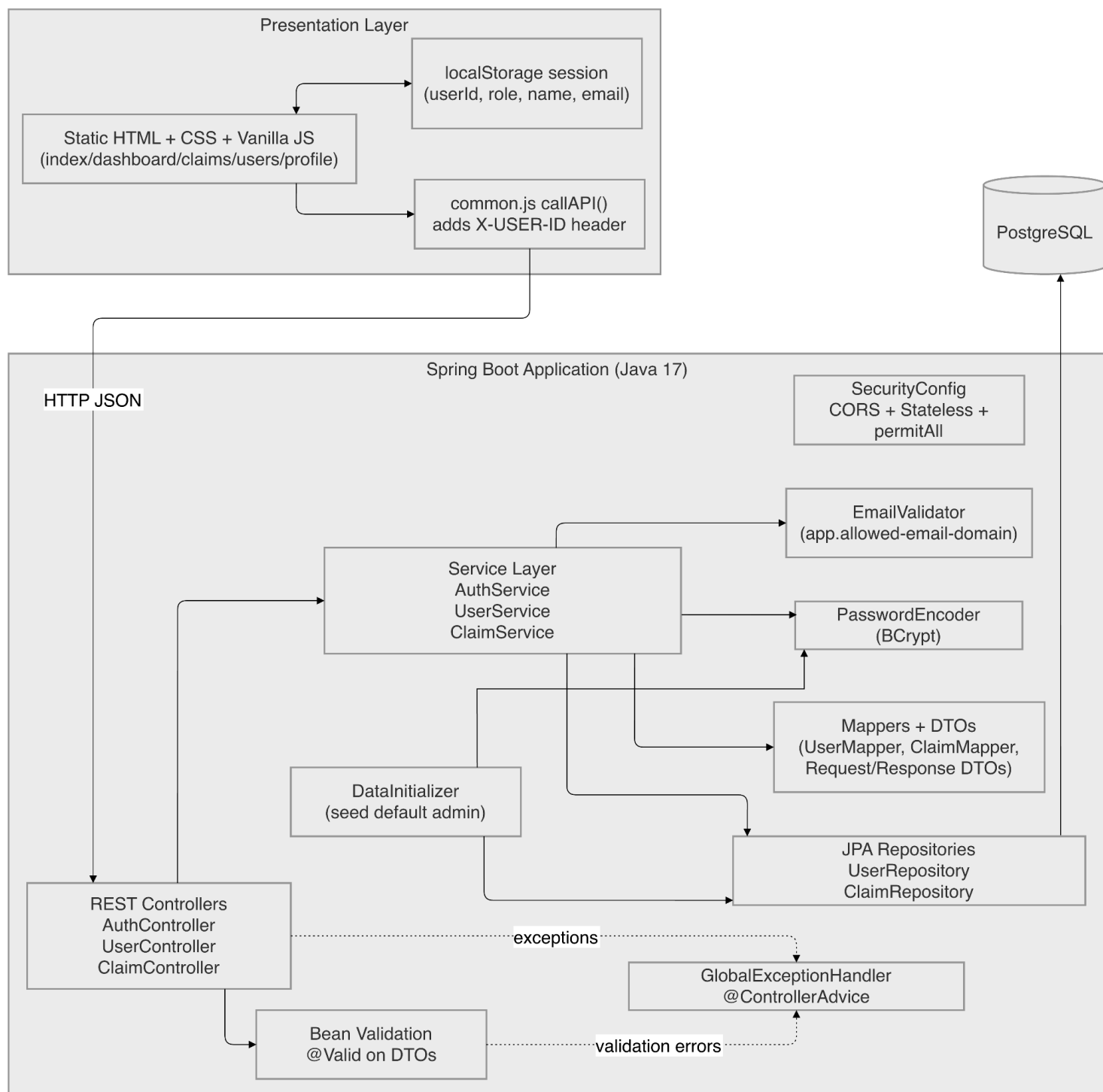
2) Claim submission workflow (employee)



3) Review workflow (manager/admin)



Architecture Diagram



Key configuration knobs

In backend/.../application.properties :

- app.allowed-email-domain : restricts registration emails (backend enforced)
- app.cors.allowed-origins : allows static frontends to call the API
- app.max-claim-amount : validation limit
- app.default-admin.* : default admin seeding
- spring.jpa.hibernate.ddl-auto=update : auto schema management